

Runbook — คู่มือใช้งานโปรเจกต์ด้วยตัวเอง

คู่มือเล่มเดียวจบสำหรับคนที่ต้อง เทรนโมเดลเอง และ เปิดหน้าบ้าน + หลังบ้านเอง บนเครื่องตัวเอง โดยไม่ต้องอ่านโค้ดก่อน

เขียนจากคำสั่งที่รันจริงบนเครื่อง Windows 11 + RTX 4060 Laptop ทุกคำสั่งในเล่มนี้ ถูกรันแล้วอย่างน้อยหนึ่งครั้ง ตัวเลขเวลาที่อ้างถึงมาจาก log จริงใน results/

0. คู่มือเล่มนี้ต่างจากเล่มอื่นอย่างไร

โปรเจกต์นี้มีเอกสารอยู่หลายเล่ม อ่านผิดเล่มจะรันคำสั่งผิดไปป์ไลน์

เอกสาร	ครอบคลุมอะไร	ใช้เมื่อไหร่
docs/runbook.md (เล่มนี้)	ปฏิบัติการทั้งวงจร: setup → เทรน 2018 → analyze → เปิด API + web	ทำงานประจำวัน / เปิดโปรเจกต์ใหม่บนเครื่องใหม่
docs/training_guide.md	เทรนฝั่ง CICIDS2017 ผ่าน main.py	ทำงานกับ 2017
docs/dashboard_guide.md	รายละเอียด dashboard + การ deploy	ปรับแต่ง dashboard เชิงลึก
src/ids2018/README.md	เหตุผลเชิงเทคนิคของไปป์ไลน์ 2018 (ทำไมสุมแบบนี้ ทำไมตัดคอลัมน์นี้)	ต้องอธิบาย/ปกป้องวิธีการ
README.md	ภาพรวมโปรเจกต์ทั้งหมด	ปฐมนิเทศ

เรื่องที่ต้องรู้ก่อนอย่างอื่น: โปรเจกต์นี้มีไปป์ไลน์แยกขาดกัน 2 ชุด

```
CICIDS2017          → main.py          → results/cicids2017_temporal_v1/
CSE-CIC-IDS2018     → python -m src.ids2018.* → results/ids2018/<tag>/
```

สองชุดนี้ ไม่แชร์โค้ด preprocessing กันเลย เพราะชื่อคอลัมน์คนละแบบ (Tot Fwd Pkts VS Total Fwd Packets) เอาคำสั่งของฝั่งหนึ่งไปรันอีกฝั่งไม่ได้

1. เตรียมเครื่อง (ทำครั้งเดียว)

1.1 สิ่งที่ต้องมี

อย่าง	เวอร์ชัน	หมายเหตุ
Python	3.10+ (เครื่องอ้างอิงใช้ 3.13.12)	สำหรับเทรน + API
Node.js	18+ (เครื่องอ้างอิงใช้ 24.13.0)	สำหรับหน้าบ้าน Next.js
RAM	16 GB ขึ้นไป	500k แถว × 69 features ใช้ ~138 MB แต่ stacking กินหลาย GB ระหว่าง fit
ดิสก์ว่าง	~5 GB	dataset อยู่คนละไดรฟ์ได้ ส่วน artefacts ต่อบันทึกราว 280 MB
GPU (ไม่บังคับ)	NVIDIA + CUDA	เร่งได้แค่ XGBoost / CatBoost / stacking

1.2 Python environment

```
cd C:\Users\ks\Documents\GitHub\ai-classification-cyber-attack
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

ตรวจว่าใช้ได้:

```
python -c "import xgboost, lightgbm, catboost, sklearn; print('ok')"
```

> ทุกคำสั่ง Python ในคู่มือนี้สมมติว่า activate venv แล้ว ถ้าไม่อย่าง activate > ให้เรียก
.\.venv\Scripts\python.exe แทนคำว่า python ตรง ๆ ได้เลย

1.3 Dataset ดิบของ CSE-CIC-IDS2018

ต้องมีไฟล์ CSV รายวัน 10 ไฟล์ (~6.7 GB) วางไว้ที่ใดเรกทอรีเดียว บนเครื่องอ้างอิงคือ D:\CSE-CIC-IDS2018

```
ls D:\CSE-CIC-IDS2018\*.csv
```

ต้องเห็นครบ 10 ไฟล์:

```
02-14-2018.csv 02-15-2018.csv 02-16-2018.csv 02-20-2018.csv 02-21-2018.csv
02-22-2018.csv 02-23-2018.csv 02-28-2018.csv 03-01-2018.csv 03-02-2018.csv
```

ถ้าเก็บไว้ที่อื่น เปลี่ยนค่า --raw-dir ในทุกคำสั่งข้างล่างให้ตรง

> 02-20-2018.csv ใหญ่ 4 GB และมี schema ต่างจากไฟล์อื่น (มี Flow ID, Src IP, > Src Port, Dst IP
เพิ่มมา 4 คอลัมน์) โค้ดจัดการให้แล้ว — อย่าแก้ไฟล์เอง

1.4 Node สำหรับที่บ้าน

```
cd web
npm install
cd ..
```

2. แผนที่โฟลเดอร์ที่ต้องรู้จัก

```
data/ids2018/
├── corpus_label_index.npz          pass-1          corpus 16.2M
├── sample_300k.parquet            sample
└── sample_500k.parquet

results/ids2018/<tag>/           metrics + log ( git)
models/ids2018/<tag>/             .joblib      ( git      .gitignore
                                metadata.json)
```

<tag> ถูกตั้งชื่ออัตโนมัติจาก ขนาด + protocol (train_ids2018.py:230):

คำสั่งที่รัน	tag ที่ได้
--sample-size 300000	300k
--sample-size 500000	500k
--sample-size 500000 --split temporal	500k_temporal
--sample-size 500000 --split temporal --tune	500k_temporal_tuned

3. กฎข้อเดียวที่ห้ามพลาด — ผลเก่าต้องไม่หาย

ระบบตั้งชื่อโฟลเดอร์ตาม tag อยู่แล้ว การรันขนาดใหม่หรือ protocol ใหม่จึงเป็น "โฟลเดอร์พี่น้อง"
ไม่ใช่การเขียนทับ รัน 500k ก็รอบก็ไม่แตะ 300k

สิ่งที่ จะ ทับของเก่าได้ มีแค่ 3 กรณีนี้:

การกระทำ	ผลที่ตามมา
รัน tag เดิมซ้ำ โดยไม่ใส่ --resume	โมเดลใน tag นั้นถูกเทรนใหม่ทับทั้งหมด
ใส่ --output-dir / --models-dir เองชี้ไปโฟลเดอร์ที่มีของอยู่	ทับ
--rebuild-sample	สร้าง parquet ของขนาดนั้นใหม่ (ผลเก่ายังอยู่ แต่ถ้า seed เปลี่ยนจะเทียบกันผลเก่าไม่ได้)

วิธีปฏิบัติที่ปลอดภัย

1. ใส่ --resume เสมอเวลารัน tag ที่เคยรันแล้ว — โมเดลที่มี artefact อยู่จะถูกข้าม
2. ก่อนรันยาว ๆ commit ผลเก่าไว้ก่อน: `git add results/ && git commit -m "wip"`
3. เช็ก tag ที่จะได้ก่อนรันจริงด้วย --dry-run — log บรรทัดที่ 4 บอกปลายทางชัดเจน:

`results -> ...\results\ids2018\500k | models -> ...\models\ids2018\500k`

4. เทรนโมเดล CSE-CIC-IDS2018 ด้วยตัวเอง

4.1 ขั้นที่ 0 — dry-run ก่อนเสมอ

ทำครบทุกขั้นยกเว้นการเทรน ใช้ยืนยันว่า data path ถูก และได้เห็นสัดส่วนคลาสที่จะได้

```
python -m src.ids2018.train_ids2018 `
  --raw-dir D:/CSE-CIC-IDS2018 `
  --sample-size 500000 `
  --sampling nested `
  --seed 42 `
  --dry-run
```

ครั้งแรกของแต่ละขนาดจะใช้เวลา ~3 นาที (pass 2 อ่าน CSV 6.7 GB เพื่อดึงแถวที่เลือก)

ครั้งต่อไปข้ามทันทีเพราะอ่านจาก sample_500k.parquet

ครั้งสุดท้ายของทั้งเครื่อง จะช้ากว่านั้นมาก เพราะต้องสแกน label ทั้ง corpus ก่อน (pass 1) แต่ผลถูกแคชที่ corpus_label_index.npz และใช้ซ้ำได้ทุกขนาด

4.2 เทรนจริง — protocol แบบสุ่ม (random stratified)

นี่คือ protocol เดียวกับบันเดิล 300k และ 500k

```
python -m src.ids2018.train_ids2018 `
  --raw-dir D:/CSE-CIC-IDS2018 `
  --sample-size 500000 `
  --sampling nested `
  --seed 42 `
  --split stratified `
  --accelerator gpu `
  --resume
```

4.3 เทรนจริง — protocol แบบเรียงเวลา (temporal)

แบ่ง 70/30 ตามลำดับเวลาจริงภายในแต่ละกลุ่ม (วันที่เก็บข้อมูล × คลาส) ไม่มี flow ในชุดทดสอบที่เกิดก่อน flow ฝึกของคลาสตัวเอง

```
python -m src.ids2018.train_ids2018 `
  --raw-dir D:/CSE-CIC-IDS2018 `
  --sample-size 500000 `
  --sampling nested `
```

```
--seed 42 `
--split temporal `
--min-test-per-class 1 `
--accelerator gpu `
--resume
```

> Timestamp เป็นแค่คีย์เรียงลำดับ ไม่เคยเป็น feature — ถูกตัดทิ้งทันทีหลัง split > และมี assert_timestamp_absent บังคับไว้ เรื่องนี้ไม่ใช่ข้อกังวลเชิงทฤษฎี: > วัตถุจริงบน corpus นี้แล้ว การปล่อยให้ timestamp รั่วดัน LightGBM macro-F1 > จาก 0.6870 ขึ้นเป็น 0.8071 — สูงกว่าผลที่ข้อสัณยทุกตัวในโปรเจกต์

4.4 ชั้นที่ขาดไม่ได้ — analyze

train_ids2018 เขียนแค่ metrics พื้นฐาน dashboard ต้องการอีก 4 ไฟล์ที่มีเฉพาะชั้นนี้ ข้ามชั้นนี้ = หน้าเว็บจะโชว์ false-alarm rate / missed attacks เป็นช่องว่าง

```
python -m src.ids2018.analyze --bundle 500k
python -m src.ids2018.analyze --bundle 500k_temporal
```

ได้ไฟล์เพิ่ม:

ไฟล์	เนื้อหา
extended_metrics.csv	MCC, false-alarm rate, missed attacks, throughput, ขนาด artefact
significance_f1_macro_ci.csv	ช่วงความเชื่อมั่น 95% ของ macro-F1 (bootstrap 1000 รอบ)
significance_mcnemar.csv	McNemar เทียบตัวนำกับคู่แข่งแบบจับคู่
per_class_f1_matrix.csv	F1 รายการสของทุกโมเดลในตารางเดียว

CI กับ McNemar ต้องใช้คำทำนาย รายแถว ซึ่งตอนเทรนไม่ได้เก็บไว้ ชั้นนี้จึงสร้าง split เดิมขึ้นมาใหม่แล้วให้โมเดลที่เซฟไว้ทำนายซ้ำ — และ ตรวจสอบว่าสร้างเหมือนเดิมจริง โดยเทียบ confusion matrix ที่ละช่องกับที่ตอนเทรนเขียนไว้ ถ้าไม่ตรงจะหยุดทันที ไม่ปล่อยให้ตัวเลขผิดพลาดออกไป

4.5 ตัวเลือกที่ใช้บ่อย

```
#
python -m src.ids2018.train_ids2018 --models xgboost lightgbm catboost

# hyperparameter search ( - search )
python -m src.ids2018.train_ids2018 --split temporal --tune --tune-iter 20 --tune-rows 60000

# ( MLP )
python -m src.ids2018.train_ids2018 --class-weighting balanced

#
python -m src.ids2018.train_ids2018 --help
```

4.6 GPU ช่วยตรงไหนบ้าง

โมเดล	--accelerator gpu
XGBoost	☐ device="cuda"
CatBoost	☐ task_type="GPU"
Stacking	☐ เฉพาะ base learner ที่เป็น XGBoost
LightGBM / RF / MLP / LogReg	☐ CPU อย่างเดียว

ขั้น preprocessing เป็น CPU ล้วน ก่อนเทรนโค้ดจะ ทดสอบ CUDA จริง (fit ปัญหาเล็ก ๆ บนการ์ด) ถ้าไม่ผ่านจะ raise ทันที ไม่แอบตกไปใช้ CPU แล้วรายงานว่าเป็นเวลาของ GPU

5. เปิดหลังบ้าน (FastAPI)

```
.\.venv\Scripts\Activate.ps1
uvicorn api.main:app --reload --port 8000
```

ตรวจว่าขึ้นแล้ว — เปิด <http://localhost:8000/docs> หรือ:

```
curl http://localhost:8000/api/bundles
```

ควรได้ JSON ที่มี id ของทุกบันเดิลใน results/ รวมทั้งตัวใหม่ที่เพิ่งเทรน

backend ค้นหาบันเดิลจากดิสก์เอง (src/bundles/registry.py:149 เดิน results/ ทีละโฟลเดอร์) จึงไม่มีไฟล์ config ที่ต้องแก้ด้วยมือเวลาเพิ่มบันเดิลใหม่ — เทรนเสร็จ + analyze เสร็จ แล้ว restart uvicorn ก็ขึ้นเอง

6. เปิดหน้าบ้าน (Next.js)

เปิด terminal ที่สอง (อันแรกปล่อยให้ uvicorn รันค้างไว้):

```
cd web
npm run dev
```

เปิด <http://localhost:3000>

หน้าบ้านอิงไปที่ <http://localhost:8000> เป็นค่าเริ่มต้น (web/src/lib/api.ts:1) ถ้าย้าย backend ไปพอร์ตอื่น ให้สร้าง web/.env.local:

```
NEXT_PUBLIC_API_URL=http://localhost:8001
```

แล้ว restart npm run dev (ตัวแปร NEXT_PUBLIC_* ถูกฝังตอน build ไม่ใช่ตอนรัน)

ลำดับที่ถูกต้อง

1.	python -m src.ids2018.train_ids2018 ...	
2. analyze	python -m src.ids2018.analyze --bundle <tag>	
3. backend	uvicorn api.main:app --reload --port 8000	(terminal 1)
4. frontend	cd web ; npm run dev	(terminal 2)
5. browser	http://localhost:3000	

ข้อ 3 กับ 4 สลับกันได้ แต่ 1 → 2 ห้ามสลับ และห้ามข้าม 2

7. ตรวจว่าบันเดิลใหม่ขึ้นจริง

```
# 1. - extended_metrics.csv significance_*.csv
ls results\ids2018\500k_temporal
```

```
# 2. backend
curl http://localhost:8000/api/bundles
```

```
# 3. - sidebar
```

ถ้าหน้าเว็บยังไม่เห็นบันเดิลใหม่: restart uvicorn (backend cache ผลไว้)

8. เวลาที่ใช้จริง

วัดบน RTX 4060 Laptop 8 GB + Windows 11 เทรนครบ 7 โมเดล จาก log ใน results/

งาน	เวลา
pass 1 สแกน label ทั้ง corpus 16.2M แถว (ครั้งแรกของเครื่องเท่านั้น)	38 วินาที
pass 2 สร้าง sample 300k จาก CSV	2.8 นาที
pass 2 สร้าง sample 500k จาก CSV	3.5 นาที
เทรน 300k, random split, GPU	8.8 นาที
เทรน 300k, temporal split, CPU	13.3 นาที
เทรน 500k, random split, GPU	9.3 นาที
analyze บันทึกลับ 300k	~1 นาที
analyze บันทึกลับ 500k	2.3 นาที (bootstrap 1000 รอบบน test 150k แถว)
hyperparameter search (--tune) 300k	~2-3 ชั่วโมง

เวลาเทรนแทบไม่ขึ้นตามขนาดข้อมูลแบบเชิงเส้น เพราะตัวที่กินเวลามากจริงคือ stacking (refit base learner cv+1 ครั้ง) ซึ่งใช้ 190 วินาทีที่ 300k และ 205 วินาทีที่ 500k

9. ปัญหาที่เจอบ่อย

อาการ	สาเหตุ	วิธีแก้
No CSV files found in ...	--raw-dir ผิด	ตรวจสอบด้วย ls D:\CSE-CIC-IDS2018*.csv
หน้าเว็บโชว์ "no bundle"	backend ไม่ได้เปิด หรือเปิดคนละพอร์ต	curl http://localhost:8000/api/bundles
false-alarm rate เป็นช่องว่าง	ยังไม่ได้รัน analyze	python -m src.ids2018.analyze --bundle <tag>
analyze ขึ้น "confusion matrix does not match"	sample parquet ถูกสร้างใหม่ด้วย seed อื่นหลังเทรน	เทรน tag นั้นใหม่ หรือกู้ parquet เดิมกลับมา
port 3000 ไม่ว่าง	มีอย่างอื่นใช้อยู่	npm run dev -- -p 3001
port 8000 ไม่ว่าง	มีอย่างอื่นใช้อยู่	uvicorn api.main:app --port 8001 + ตั้ง NEXT_PUBLIC_API_URL
CUDA verification ล้ม	ไดรเวอร์/CUDA ไม่พร้อม	ติดตั้ง --accelerator gpu ออก รันบน CPU
เทรนแล้วเข้าผิดปกติที่ stacking	stacking refit base learner cv+1 ครั้ง	ปกติ — หรือตัดออกด้วย --models ที่ไม่ใช่ stacking

10. อ่านตัวเลขอย่างไรไม่ให้หลงทาง

- ดู f1_macro ไม่ใช่ accuracy — Benign คิดเป็น ~83% ของข้อมูล โมเดลที่จับ

SQL Injection ไม่ได้เลยก็จะได้ accuracy ~0.98

- ดูช่วงความเชื่อมั่นก่อนสรุปว่าใครชนะ — ใน significance_f1_macro_ci.csv

ช่วงความเชื่อมั่นของ macro-F1 กว้างเฉลี่ย ~0.128 ที่ขนาด 300k เพราะมี 3 คลาสที่ มี test flow แค่ 1, 2 และ 4 แถว แต่ถือนำหนัก 3/15 ของ macro-F1 — flow เดียว เปลี่ยนคลาสขยับ metric ได้ถึง 0.067

- ตัวเลขที่มีความหมายเชิงปฏิบัติการคือ false-alarm rate กับ missed attacks

ไม่ใช่ headline accuracy

บทเรียนจากบันได 300k → 500k

รัน 500k แล้วได้ข้อสรุปที่ต้องรู้ก่อนตีความบันไดขนาดถัดไป:

บันได	ตัวนำ	macro-F1	ช่วงความเชื่อมั่น 95%	ความกว้าง CI เฉลี่ย
300k	stacking	0.6886	[0.676, 0.803]	0.1245
500k	lightgbm	0.7311	[0.692, 0.840]	0.1294
300k_temporal	catboost	0.7277	[0.680, 0.805]	0.1284
500k_temporal	lightgbm	0.7318	[0.691, 0.847]	0.1268

ข้อมูลเพิ่มขึ้น 67% แต่ช่วงความเชื่อมั่นไม่แคบลงเลย เพราะความกว้างของ CI ไม่ได้มาจากขนาดข้อมูลรวม แต่มาจากคลาสที่หายากที่สุด ซึ่งที่ 500k ยังมี test flow แค่ 1 (SQL Injection), 2 (Brute Force -XSS) และ 6 (Brute Force -Web) แยก ทั้งที่รวมกันก็น่าหนัก 3/15 ของ macro-F1

ดังนั้น อย่าอ่าน macro-F1 ที่ขยับขึ้นว่าเป็นผลของข้อมูลที่มากขึ้น — ภายใต random split โมเดล tree ทุกตัวขยับขึ้น +0.03 ถึง +0.04 ซึ่งดูเหมือนเป็นเทรนด์ แต่ภายใต temporal split ทิศทางกลับไม่สอดคล้องกัน (lightgbm +0.029 แต่ catboost -0.044 และ mlp -0.048) การขยับทั้งสองแบบเล็กกว่าความกว้าง CI มาก จึงแยกไม่ออก จากความผันผวนของคลาสหายาก

ผลที่รอดจากช่วงความเชื่อมั่นและทำซ้ำได้ มีสองอย่าง:

1. stacking มี false-alarm rate ต่ำที่สุดภายใต temporal split และค่านี้ซ้ำได้ — 0.0214% ที่ 300k และ 0.0217% ที่ 500k คราวนี้วัดบน benign 124,610 flow แทนที่จะเป็น 74,769 flow

1. Infiltration คือคลาสที่ยากจริง ไม่ใช่ปัญหาการสุม — F1 อยู่ที่

0.02–0.10 สำหรับทุกโมเดล ทั้งที่มี test flow ถึง 1,497 แยก ต่างจาก SQL Injection ที่ F1 = 0 เพราะมีแค่แถวเดียว

ถ้าจะไต่ไป 1M ต่อ ให้คาดหวังผลแบบเดียวกัน: คลาสใหญ่จะนิ่งขึ้น แต่ CI จะยังกว้าง จนกว่าจะแก้ที่ต้นเหตุ — ซึ่งไม่ใช่การเพิ่มขนาด sample แต่คือการเลิกให้คลาสที่มีตัวอย่างหลักหน่วยก็น่าหนักเท่ากับคลาสที่มีแสนแถวใน macro-F1